Okay, absolutely. Sometimes the simplest way is best! I will paste the entire macOS Terminal Command Reference Sheet directly into this chat for you. This should make it very straightforward to select and copy the text.

Here it is:

---

macOS Terminal Command Reference Sheet

This guide provides a reference for common macOS terminal commands, their uses, options, and examples.

---

1. File System Navigation

ls - List Directory Contents

- **Definition:** Lists files and directories in the current or a specified directory.
- **Why & How:** Essential for seeing what's in a directory.
  - ls (lists current directory)
  - ls [directory_path] (lists specified directory)
- **Common Options (Key Tags):**
  - -l: (long format) Displays detailed information (permissions, owner, size, date).
  - -a: (all) Shows hidden files (those starting with a .).
  - -h: (human-readable) With -l, shows sizes in a human-friendly format (KB, MB, GB).
  - -t: Sorts by modification time, newest first.
  - -R: (recursive) Lists subdirectories recursively.
- **Live Example:**
  Bash
  ```
  # List all files in detailed, human-readable format in the Documents folder, sorted by time
  ls -lath ~/Documents
  ```

cd - Change Directory

- **Definition:** Changes the current working directory.
- **Why & How:** The primary way to move between folders in the terminal.
  - cd [directory_path]
- **Special Paths:**
  - cd ~: Go to the home directory.
  - cd ..: Go to the parent directory (up one level).

    - cd -: Go to the previous directory you were in.
    - cd /: Go to the root directory.
- **Live Example:**
  Bash
  ```
  # Navigate to the Desktop, then into a folder named 'Projects', then back to home
  cd ~/Desktop
  cd Projects
  cd ~
  ```

pwd - Print Working Directory

- **Definition:** Displays the full path of the current working directory.
- **Why & How:** Useful to know exactly where you are in the file system.
    - pwd
- **Live Example:**
  Bash
  ```
  # Show the current directory path
  pwd
  # Output might be: /Users/yourusername/Documents
  ```

---

2. File & Directory Manipulation

mkdir - Make Directory

- **Definition:** Creates a new directory.
- **Why & How:** To organize files into folders.
    - mkdir [directory_name]
- **Common Options:**
    - -p: (parents) Creates parent directories as needed if they don't exist.
- **Live Example:**
  Bash
  ```
  # Create a folder named 'MyNewApp'
  mkdir MyNewApp

  # Create nested folders 'MyProject/src/components' even if 'MyProject' or 'src' don't exist
  mkdir -p MyProject/src/components
  ```

rmdir - Remove Directory

- **Definition:** Deletes an *empty* directory.

- **Why & How:** To remove unused empty folders.
  - rmdir [directory_name]
- **Note:** For non-empty directories, use rm -r.
- **Live Example:**
  Bash
  ```
  # Remove an empty directory named 'OldFolder'
  rmdir OldFolder
  ```

touch - Create Empty File / Update Timestamp

- **Definition:** Creates a new empty file if it doesn't exist, or updates the access and modification timestamps of an existing file.
- **Why & How:** Quickly create new files or "touch" existing ones (e.g., for build systems).
  - touch [filename]
- **Live Example:**
  Bash
  ```
  # Create an empty file named 'notes.txt'
  touch notes.txt

  # Update the timestamp of an existing file 'config.yml'
  touch config.yml
  ```

cp - Copy Files and Directories

- **Definition:** Copies files or directories.
- **Why & How:** To duplicate files or back them up.
  - cp [source_file] [destination_file] (copy and rename)
  - cp [source_file] [destination_directory] (copy into directory)
  - cp -r [source_directory] [destination_directory] (copy directory recursively)
- **Common Options:**
  - -r or -R: (recursive) Copies directories and their contents. **Required for directories.**
  - -v: (verbose) Shows files as they are copied.
  - -p: (preserve) Preserves attributes like modification time, ownership, and permissions.
- **Live Example:**
  Bash
  ```
  # Copy 'report.docx' to 'report_backup.docx' in the same folder
  cp report.docx report_backup.docx
  ```

```
# Copy 'image.jpg' into the 'Pictures' folder
cp image.jpg ~/Pictures/

# Copy the entire 'ProjectA' folder to 'ProjectA_Archive'
cp -r ProjectA ProjectA_Archive
```

mv - Move or Rename Files and Directories

- **Definition:** Moves files/directories to a different location, or renames them if the destination is in the same location.
- **Why & How:** To reorganize files or change their names.
  - mv [source] [destination_directory] (move)
  - mv [old_name] [new_name] (rename)
- **Common Options:**
  - -v: (verbose) Shows files as they are moved/renamed.
  - -i: (interactive) Prompts before overwriting an existing file.
- **Live Example:**

```
Bash
# Rename 'draft.txt' to 'final_report.txt'
mv draft.txt final_report.txt

# Move 'archive.zip' into the 'Backup' folder
mv archive.zip ~/Backup/

# Move all .jpg files from current directory to Pictures/Vacation
mv *.jpg ~/Pictures/Vacation/
```

rm - Remove Files or Directories

- **Definition:** Deletes files or directories. **Use with extreme caution, as deleted files are generally not recoverable from the terminal.**
- **Why & How:** To permanently delete items.
  - rm [filename] (remove a file)
  - rm -r [directory_name] (remove a directory and its contents recursively)
- **Common Options:**
  - -r or -R: (recursive) Required to delete directories.
  - -f: (force) Suppresses confirmation prompts and ignores non-existent files. **Very dangerous if misused.**
  - -i: (interactive) Prompts for confirmation before each removal.

    - -v: (verbose) Shows files as they are deleted.
- **Live Example:**
  Bash
  ```
  # Remove a file named 'temp.log'
  rm temp.log

  # Interactively remove a file 'old_script.sh'
  rm -i old_script.sh

  # Remove a directory 'UnusedFolder' and all its contents (use carefully!)
  rm -r UnusedFolder

  # Force remove 'stubborn.file' (EXTREME CAUTION!)
  # rm -f stubborn.file
  ```

---

3. Viewing File Contents

cat - Concatenate and Display File Content

- **Definition:** Displays the entire content of one or more files to the standard output (your terminal screen).
- **Why & How:** Quickly view short files or combine multiple files.
    - cat [filename]
    - cat [file1] [file2] > [newfile] (concatenate file1 and file2 into newfile)
- **Common Options:**
    - -n: Number all output lines.
    - -b: Number non-empty output lines.
- **Live Example:**
  Bash
  ```
  # Display the content of 'config.txt'
  cat config.txt

  # Display content of 'header.txt' then 'body.txt'
  cat header.txt body.txt
  ```

less - View File Content Page by Page (More Advanced)

- **Definition:** Allows you to view file content interactively, one page at a time. You can scroll up and down, search, etc.
- **Why & How:** Ideal for viewing larger files.

    - less [filename]
- **Navigation within less:**
    - Spacebar or f: Next page.
    - b: Previous page.
    - Arrow Up/Down: Scroll line by line.
    - /search_term: Search forward for search_term.
    - ?search_term: Search backward for search_term.
    - n: Repeat last search.
    - N: Repeat last search in opposite direction.
    - g: Go to the beginning of the file.
    - G: Go to the end of the file.
    - q: Quit less.
- **Live Example:**
  Bash
  # View a large log file
  less server.log

more - View File Content Page by Page (Simpler)

- **Definition:** Similar to less but simpler; primarily allows forward scrolling.
- **Why & How:** A basic pager for viewing files. less is generally preferred.
    - more [filename]
- **Navigation within more:**
    - Spacebar: Next page.
    - Enter: Next line.
    - q: Quit.
- **Live Example:**
  Bash
  # View a text file
  more story.txt

head - Display Beginning of a File

- **Definition:** Shows the first few lines of a file.
- **Why & How:** Quickly peek at the start of a file.
    - head [filename] (shows first 10 lines by default)
- **Common Options:**
    - -n [number]: Specifies the number of lines to show. (e.g., head -n 5 file.txt)
- **Live Example:**
  Bash

```
# Show the first 5 lines of 'data.csv'
head -n 5 data.csv
```

tail - Display End of a File

- **Definition:** Shows the last few lines of a file.
- **Why & How:** Useful for checking recent log entries or the end of large files.
  - tail [filename] (shows last 10 lines by default)
- **Common Options:**
  - -n [number]: Specifies the number of lines to show.
  - -f: (follow) Outputs appended data as the file grows; great for monitoring live log files. (Press Ctrl+C to stop).
- **Live Example:**

```
Bash
# Show the last 20 lines of 'access.log'
tail -n 20 access.log

# Monitor 'application.log' in real-time
tail -f application.log
```

---

4. Searching for Files & Text

find - Search for Files

- **Definition:** Searches for files and directories within a directory hierarchy based on various criteria (name, type, size, modification time, etc.).
- **Why & How:** Powerful tool for locating files.
  - find [path_to_search] -name "[filename_pattern]"
  - find [path_to_search] -type [f/d] (f for file, d for directory)
- **Common Options & Tests:**
  - . (dot): Represents the current directory.
  - -name "pattern": Search by filename (e.g., *.txt). Use quotes for patterns.
  - -iname "pattern": Case-insensitive name search.
  - -type f: Find files.
  - -type d: Find directories.
  - -mtime -[days]: Find files modified within the last [days] days (e.g., -mtime -7 for last 7 days).
  - -mtime +[days]: Find files modified more than [days] days ago.
  - -size +[size]c/k/M/G: Find files larger than [size] (c=bytes, k=KB, M=MB,

G=GB). (e.g. +10M)
    - -exec [command] {} \;: Execute [command] on each found file. {} is replaced by the filename. \; terminates the command.
- **Live Example:**

```Bash
# Find all .log files in the current directory and its subdirectories
find . -name "*.log"

# Find all directories named 'src' under the home directory
find ~ -type d -name "src"

# Find all .jpg files larger than 1MB in ~/Pictures and list them
find ~/Pictures -type f -name "*.jpg" -size +1M -ls

# Find all .tmp files in /var/log modified in the last 2 days and delete them (CAREFUL!)
# find /var/log -name "*.tmp" -mtime -2 -exec rm {} \;
```

grep - Search for Text Patterns in Files

- **Definition:** Searches for lines containing a match to a specified pattern (text string or regular expression) in files or input streams.
- **Why & How:** Invaluable for finding specific text within files.
    - grep "[pattern]" [filename(s)]
- **Common Options:**
    - -i: (ignore case) Case-insensitive search.
    - -r or -R: (recursive) Search recursively in directories.
    - -l: (files with match) Only print filenames of files containing matches.
    - -c: (count) Print count of matching lines per file.
    - -v: (invert match) Select non-matching lines.
    - -n: (line number) Prefix each line of output with its line number in the input file.
    - -E: Use extended regular expressions.
- **Live Example:**

```Bash
# Search for the word "error" in 'server.log'
grep "error" server.log

# Case-insensitive search for "warning" in all .txt files in the current directory
grep -i "warning" *.txt

# Recursively search for "API_KEY" in all files under the 'my_project' directory
```

```
grep -r "API_KEY" my_project/

# Find files containing "TODO" and list only the filenames
grep -rl "TODO" .

# Count lines containing "success" in 'results.txt'
grep -c "success" results.txt
```

- **Piping to grep:** grep is often used with other commands via a pipe (|).
  ```
  Bash
  # List processes and filter for those related to "chrome"
  ps aux | grep -i "chrome"
  ```

---

5. Permissions

chmod - Change File Mode (Permissions)

- **Definition:** Modifies the access permissions of files and directories.
- **Why & How:** To control who can read, write, or execute files.
- **Modes:**
  - **Symbolic:** Uses letters (u=user/owner, g=group, o=others, a=all; +=add, -=remove, ==set; r=read, w=write, x=execute).
    - chmod u+x [filename] (make executable for user)
    - chmod go-w [filename] (remove write permission for group and others)
  - **Octal (Numeric):** Uses numbers (4=read, 2=write, 1=execute). Sum them for each category (user, group, others).
    - rwx = 4+2+1 = 7
    - rw- = 4+2+0 = 6
    - r-x = 4+0+1 = 5
    - r-- = 4+0+0 = 4
    - chmod 755 [filename] (user:rwx, group:r-x, others:r-x - common for scripts)
    - chmod 644 [filename] (user:rw-, group:r--, others:r-- - common for regular files)
- **Common Options:**
  - -R: (recursive) Apply changes to directories and their contents.
- **Live Example:**
  ```
  Bash
  # Make 'myscript.sh' executable by the owner
  chmod u+x myscript.sh
  ```

```
# or using octal:
chmod 700 myscript.sh # (rwx------)

# Give read and write permission to owner, and read-only to group and others for 'config.txt'
chmod 644 config.txt

# Recursively remove write permission for 'others' from the 'SharedFolder'
chmod -R o-w SharedFolder
```

chown - Change File Owner and Group

- **Definition:** Changes the user and/or group ownership of a file or directory.
- **Why & How:** To transfer ownership. Usually requires sudo (administrator privileges).
  - sudo chown [new_owner] [filename]
  - sudo chown :[new_group] [filename] (note the colon before group name)
  - sudo chown [new_owner]:[new_group] [filename]
- **Common Options:**
  - -R: (recursive) Apply changes to directories and their contents.
- **Live Example:**

```
Bash
# Change owner of 'data.file' to 'jane' (assuming 'jane' is a valid username)
sudo chown jane data.file

# Change group of 'report.doc' to 'editors'
sudo chown :editors report.doc

# Change owner to 'john' and group to 'developers' for 'app_source' directory recursively
sudo chown -R john:developers app_source/
```

---

6. Networking

ping - Send ICMP ECHO_REQUEST to Network Hosts

- **Definition:** Tests network connectivity to a host by sending packets and waiting for replies.
- **Why & How:** Basic network troubleshooting to see if a server is reachable.
  - ping [hostname_or_ip]
- **Common Options (macOS):**
  - -c [count]: Stop after sending [count] packets.

- **Live Example:**
  Bash
  ```
  # Ping google.com 5 times
  ping -c 5 google.com
  ```

ifconfig (Older) / ip (Newer Linux, ifconfig still common on macOS for basic info)

- **Definition:** ifconfig is used to configure (or view the configuration of) network interfaces. On modern Linux, ip addr is preferred, but ifconfig is still present on macOS for quick checks.
- **Why & How:** To see your IP address, MAC address, and other network interface details.
  - ifconfig (shows all active interfaces)
  - ifconfig en0 (shows details for interface en0, often Wi-Fi or Ethernet)
- **Live Example:**
  Bash
  ```
  # Display configuration for all network interfaces
  ifconfig

  # Display configuration for the Wi-Fi interface (usually en0 or en1)
  ifconfig en0
  ```

netstat - Network Statistics

- **Definition:** Displays network connections, routing tables, interface statistics, etc.
- **Why & How:** To see active network connections and listening ports.
- **Common Options (macOS):**
  - -an: Display all active TCP connections and listening TCP and UDP ports numerically.
  - -r: Display routing table.
  - -p tcp: Show TCP connections.
- **Live Example:**
  Bash
  ```
  # Show active TCP connections and listening ports
  netstat -p tcp -an | grep LISTEN

  # Display the routing table
  netstat -r
  ```

curl - Transfer Data from or to a Server

- **Definition:** A command-line tool for transferring data using various protocols (HTTP, HTTPS, FTP, etc.).
- **Why & How:** Download files, test APIs, view HTTP headers.
  - curl [URL]
- **Common Options:**
  - -O: (output) Save downloaded content to a file with the same name as the remote file.
  - -o [filename]: Save downloaded content to a specified [filename].
  - -L: (location) Follow redirects.
  - -I: (head) Show document information/headers only (HEAD request).
  - -X [METHOD]: Specify HTTP method (e.g., POST, PUT, DELETE).
  - -H "[Header: Value]": Add a custom HTTP header.
  - -d "[data]": Send HTTP POST data.
- **Live Example:**

```
Bash
# Fetch the content of a webpage and display it
curl https://example.com

# Download a file and save it as 'archive.zip'
curl -o archive.zip https://example.com/files/archive.zip

# Get only the HTTP headers from a URL
curl -I https://api.example.com/status

# Send a POST request with JSON data
curl -X POST -H "Content-Type: application/json" -d '{"name":"test"}'
https://api.example.com/users
```

ssh - Secure Shell Client

- **Definition:** Connects securely to a remote machine over a network.
- **Why & How:** To log in and execute commands on a remote server.
  - ssh [username]@[hostname_or_ip]
  - ssh -p [port] [username]@[hostname_or_ip] (if SSH is on a non-standard port)
- **Common Options:**
  - -p [port]: Specify the port to connect to on the remote host.
  - -i [path_to_private_key]: Specify an identity file (private key) to use for

authentication.
    - -v: Verbose mode, useful for debugging connection issues.
- **Live Example:**

```
Bash
# Connect to a remote server as user 'admin'
ssh admin@yourserver.com

# Connect using a specific private key
ssh -i ~/.ssh/my_custom_key user@anotherhost.com
```

---

7. Process Management

ps - Report a Snapshot of Current Processes

- **Definition:** Displays information about currently running processes.
- **Why & How:** To see what programs are running.
- **Common Options (macOS uses BSD-style options):**
    - aux: (a=all users, u=user-oriented format, x=include processes without a controlling terminal) A common combination to see all processes.
    - ef: (e=all processes, f=full format) Another common way, more standard on Linux but works on macOS.
- **Live Example:**

```
Bash
# Show all running processes with detailed user-oriented information
ps aux

# Often piped with grep to find a specific process
ps aux | grep -i "firefox"
```

kill - Terminate or Signal Processes

- **Definition:** Sends a signal to a process, typically to terminate it.
- **Why & How:** To stop misbehaving or unwanted processes.
    - kill [PID] (sends TERM signal by default, allowing graceful shutdown)
    - kill -9 [PID] or kill -SIGKILL [PID] (sends KILL signal, forces immediate termination - use as last resort)
- **Finding the PID (Process ID):** Use ps aux | grep [process_name]. The PID is usually the second column.
- **Common Signals:**

    - TERM (15): Default. Polite request to terminate.
    - KILL (9): Forceful termination. Data loss can occur.
    - HUP (1): Hang up. Often used to tell daemons to re-read config files.
- **Live Example:**

```Bash
# Assume 'my_app' has PID 12345
# Politely ask 'my_app' to terminate
kill 12345

# Forcefully terminate 'my_app' if it's unresponsive (use with caution)
kill -9 12345
```

top - Display Dynamic Real-time View of Processes

- **Definition:** Provides a continuously updated list of running processes, typically sorted by CPU usage.
- **Why & How:** To monitor system activity and resource usage in real-time.
    - top
- **Navigation within top (macOS version):**
    - q: Quit.
    - o: Change sort order (e.g., type cpu to sort by CPU, mem for memory).
    - spacebar: Update immediately.
- **Live Example:**

```Bash
# Start top to monitor system processes
top
# (Press 'q' to exit)
```

---

8. System Information

uname - Print System Information

- **Definition:** Displays information about the machine and operating system.
- **Why & How:** To quickly identify the OS type and kernel version.
    - uname
- **Common Options:**
    - -a: (all) Print all available information.
    - -m: Machine hardware name (e.g., arm64, x86_64).
    - -s: Kernel name (e.g., Darwin).
- **Live Example:**

```
Bash
# Show all system information
uname -a
# Output might be: Darwin Your-MacBook-Pro.local 23.1.0 Darwin Kernel Version 23.1.0: Mon Oct 9 21:27:27 PDT 2023; root:xnu-10002.41.9~6/RELEASE_ARM64_T8103 arm64
```

df - Report File System Disk Space Usage

- **Definition:** Shows the amount of disk space used and available on file systems.
- **Why & How:** To check free disk space.
  - df
- **Common Options:**
  - -h: (human-readable) Display sizes in powers of 1024 (KB, MB, GB).
  - -H: (human-readable, SI) Display sizes in powers of 1000.
- **Live Example:**

```
Bash
# Show disk space usage in human-readable format
df -h
```

du - Estimate File Space Usage

- **Definition:** Shows the disk space used by files and directories.
- **Why & How:** To find out which files/folders are taking up the most space.
  - du [path]
- **Common Options:**
  - -h: (human-readable) Display sizes in a human-friendly format.
  - -s: (summarize) Display only a total for each argument.
  - -d [depth] or --max-depth=[depth]: Display total for a directory (or file, with s) only if it is N or fewer levels below the command line argument.
- **Live Example:**

```
Bash
# Show disk usage of the current directory and its subdirectories, human-readable
du -h

# Show a summary of the disk usage for the 'Documents' folder
du -sh ~/Documents

# Show disk usage for items in the current directory, up to 1 level deep
du -h -d 1 .
```

history - Display Command History

- **Definition:** Shows a list of previously executed commands.
- **Why & How:** To recall or re-run past commands.
  - history
- **Usage:**
  - ![number]: Execute the command at that number from history (e.g., !101).
  - !!: Execute the last command.
  - !string: Execute the most recent command starting with string.
  - Ctrl+R: Reverse search through history. Start typing, and it will find matches.
- **Live Example:**

```Bash
# Display command history
history

# Display last 10 commands
history 10

# Re-run command number 50 from history
# !50
```

---

9. Archive & Compression

tar - Archiving Utility

- **Definition:** Creates, views, or extracts tar archives (tarballs). Often used with compression like gzip or bzip2.
- **Why & How:** To bundle multiple files/directories into a single archive file, often for backup or distribution.
- **Common Operations:**
  - c: Create an archive.
  - x: Extract from an archive.
  - t: List contents of an archive.
  - v: Verbose output (show files being processed).
  - f [filename]: Specify archive filename. **This option usually comes last.**
  - z: Filter through gzip (for .tar.gz or .tgz).
  - j: Filter through bzip2 (for .tar.bz2).
- **Live Example:**

```Bash
# Create a gzipped tar archive of 'MyFolder' named 'archive.tar.gz'
```

```
tar -cvzf archive.tar.gz MyFolder/

# List contents of 'archive.tar.gz'
tar -tvf archive.tar.gz

# Extract 'archive.tar.gz' into the current directory
tar -xvzf archive.tar.gz

# Create a bzip2 compressed tar archive
# tar -cvjf archive.tar.bz2 MyFolder/

# Extract a bzip2 compressed tar archive
# tar -xvjf archive.tar.bz2
```

gzip - Compress or Expand Files

- **Definition:** Compresses files using Lempel-Ziv coding (LZ77). Typically adds a .gz extension.
- **Why & How:** To reduce file size.
  - gzip [filename] (compresses, replaces original with filename.gz)
- **Common Options:**
  - -d: Decompress (same as gunzip).
  - -k: Keep original file.
- **Live Example:**

```
Bash
# Compress 'largefile.txt' to 'largefile.txt.gz' (original is removed)
gzip largefile.txt

# Compress 'data.log' and keep the original 'data.log'
gzip -k data.log
```

gunzip - Decompress Files Compressed by gzip

- **Definition:** Decompresses files with a .gz extension.
- **Why & How:** To restore gzipped files to their original state.
  - gunzip [filename.gz] (replaces filename.gz with filename)
- **Live Example:**

```
Bash
# Decompress 'archive.txt.gz' to 'archive.txt'
gunzip archive.txt.gz
```

zip - Package and Compress Files

- **Definition:** Creates ZIP archives.
- **Why & How:** Common cross-platform archiving format.
  - zip [archive_name.zip] [file1] [file2] [directory1]
- **Common Options:**
  - -r: Recursively include directory contents.
- **Live Example:**
  Bash
  ```
  # Create 'my_archive.zip' containing 'file.txt' and the 'Photos' directory
  zip -r my_archive.zip file.txt Photos/
  ```

unzip - Extract Files from a ZIP Archive

- **Definition:** Extracts files from ZIP archives.
- **Why & How:** To access contents of a ZIP file.
  - unzip [archive_name.zip]
- **Common Options:**
  - -d [destination_directory]: Extract files into a specific directory.
- **Live Example:**
  Bash
  ```
  # Extract 'my_archive.zip' into the current directory
  unzip my_archive.zip

  # Extract 'website_files.zip' into a folder named 'MySite'
  unzip website_files.zip -d MySite/
  ```

---

10. SSH Key Management

ssh-keygen - Authentication Key Generation

- **Definition:** Generates, manages, and converts authentication keys for SSH.
- **Why & How:** To create SSH key pairs (public and private) for passwordless authentication to servers.
  - ssh-keygen (interactive, will prompt for options)
- **Common Options:**
  - -t [type]: Specifies the type of key to create (e.g., rsa, ed25519 - ed25519 is generally recommended).

    - -b [bits]: Specifies the number of bits in the key (e.g., 4096 for RSA).
    - -C "[comment]": Provides a comment, often your email address.
    - -f [filename]: Specifies the filename of the key file (default is ~/.ssh/id_rsa or ~/.ssh/id_ed25519).
- **Live Example:**

```
Bash
# Generate an Ed25519 SSH key pair with a comment
ssh-keygen -t ed25519 -C "your_email@example.com"
# Follow prompts for file location (default is usually fine) and passphrase (optional but recommended).

# Generate an RSA 4096-bit key pair
# ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
```

    - This creates two files:
        - ~/.ssh/id_ed25519 (or id_rsa): Your **private key**. Keep this secret and secure!
        - ~/.ssh/id_ed25519.pub (or id_rsa.pub): Your **public key**. This is what you share/upload.

Displaying Your Public Key

- **Definition:** To view the content of your public SSH key so you can copy it.
- **Why & How:** Needed when setting up SSH access to servers (like GitHub, AWS CodeCommit, EC2 instances).
- **Command:**

```
Bash
cat ~/.ssh/id_ed25519.pub
# or if you used RSA:
# cat ~/.ssh/id_rsa.pub
```

- **Live Example:**

```
Bash
# Display your Ed25519 public key
cat ~/.ssh/id_ed25519.pub
# Copy the entire output (starts with 'ssh-ed25519 ...' or 'ssh-rsa ...')
```

This sheet covers many of the most frequently used commands. The terminal is a powerful tool, and each command often has many more options. Use man [command_name] (e.g., man ls) to access the manual page for any command and see

all its details. Happy commanding!